UNIVERSIDAD CENFOTEC

Proyecto de Maestría

Diseño de una Plataforma MLOps para Predicción de Duración de Viajes de NYC Yellow Taxi

Estructura propuesta y revisión técnica - Versión 1.0

Programa	Maestría en Analítica de Datos
Curso	Tópicos Especiales en Analítica de Datos
Estudiantes	Bernal Rojas Jorge Garnier Jorge Bermúdez
Dataset	NYC Taxi & Limousine Commission - Yellow Taxi Trip Records
Fecha	Septiembre de 2026

Resumen ejecutivo

Propósito. Diseñar, implementar y demostrar un ciclo MLOps reproducible de extremo a extremo utilizando los registros públicos de taxis amarillos de la NYC Taxi & Limousine Commission (TLC). El proyecto cubrirá ingesta mensual, validación de calidad, preparación de datos, entrenamiento, registro y versionado del modelo, despliegue de una API, contenerización, integración continua, monitoreo y reentrenamiento controlado.

Caso de uso propuesto. Predecir la duración de un viaje en minutos al momento de iniciarlo, usando únicamente variables disponibles o razonablemente conocidas en ese instante. La variable objetivo se derivará como la diferencia entre tpep_dropoff_datetime y tpep_pickup_datetime. Este enfoque permite incorporar una política explícita contra data leakage y evaluar el comportamiento del modelo en el tiempo.

Revisión clave: La propuesta original de usar Python, Docker, FastAPI y GitHub es sólida, pero para que el trabajo sea realmente MLOps se recomienda añadir herramientas concretas para tracking/registro y monitoreo. En esta estructura se propone MLflow para experimentos y Model Registry, GitHub Actions para CI/CD y Evidently para monitoreo de datos y drift. MLOps se tratará como la disciplina que integra estos componentes, no como una herramienta aislada.

1. Estructura general del proyecto

Bloque	Componente	Resultado esperado
1	Definición del problema	Pregunta analítica, usuario del modelo, alcance y criterio de éxito.
2	Ingesta y versionado de datos	Descarga parametrizada por año/mes; persistencia raw; metadatos y checksums.
3	Calidad y validación	Esquema esperado, nulos, duplicados, rangos, categorías, fechas y reglas de negocio.
4	Preparación y feature engineering	Limpieza reproducible, target, variables temporales y ruta origen-destino.
5	Entrenamiento y evaluación	Baseline, candidatos, división temporal, métricas y selección.
6	Tracking y registro	MLflow: parámetros, métricas, artefactos, lineage y versiones.
7	Serving	FastAPI con contrato de entrada/salida, validación Pydantic y endpoints de salud.
8	Contenerización y CI/CD	Docker, Docker Compose y GitHub Actions para pruebas y build.

Bloque	Componente	Resultado esperado
9	Monitoreo	Calidad, drift, rendimiento del modelo y salud del servicio.
10	Retraining y promoción	Triggers, reentrenamiento, comparación challenger/champion y rollback.
11	Documentación y reproducibilidad	README, ADRs, arquitectura, instrucciones y evidencia de ejecución.

2. Justificación del dataset y alcance de datos

La TLC publica los Trip Record Data de taxis amarillos en archivos Parquet mensuales. Los registros incluyen fechas y horas de recogida y llegada, zonas de origen y destino, distancia, tarifas, tipos de pago y conteo de pasajeros. La propia TLC indica que los datos provienen de proveedores tecnológicos y que pueden existir limitaciones de exactitud, por lo que la calidad de datos forma parte del problema y no debe tratarse como un supuesto.

El proyecto se diseñará para que el mes de procesamiento sea un parámetro, evitando acoplar el pipeline a un archivo particular. La ingesta debe tolerar cambios de esquema controlados y fallar de manera explícita ante cambios incompatibles. Desde 2025, por ejemplo, el dataset incorpora el campo cbd_congestion_fee, lo cual ilustra por qué el versionado de esquema es relevante.

Fuente oficial: NYC TLC Trip Record Data

Diccionario oficial: Yellow Taxi Trip Records - Data Dictionary

Alcance temporal recomendado: Para simular un entorno productivo sin depender del mes exacto disponible, usar 2025 como histórico principal y meses de 2026 como lotes posteriores de monitoreo. Una división inicial razonable es: entrenamiento enero-septiembre 2025; validación octubre 2025; prueba noviembre-diciembre 2025; simulación productiva por lotes con 2026. La división definitiva puede ajustarse según recursos de cómputo.

3. Objetivos

3.1 Objetivo general

Implementar una solución MLOps reproducible para predecir la duración de viajes de taxi amarillo de la ciudad de Nueva York, gestionando el ciclo de vida completo del modelo desde la ingesta de datos hasta el monitoreo y reentrenamiento, mediante Python, MLflow, FastAPI, Docker y GitHub/GitHub Actions.

3.2 Objetivos específicos

- Diseñar una ingesta parametrizada y reproducible de archivos Parquet mensuales de NYC TLC.

- Implementar Data Quality Gates automáticos antes de permitir que un lote continúe al pipeline de entrenamiento o inferencia.
- Construir un pipeline de transformación y feature engineering compartido entre entrenamiento e inferencia para evitar training-serving skew.
- Entrenar y comparar un baseline con al menos dos modelos candidatos usando validación temporal y métricas reproducibles.
- Registrar experimentos, métricas, artefactos y versiones de modelos en MLflow.
- Publicar el modelo seleccionado mediante una API REST en FastAPI y empaquetarla en Docker.
- Automatizar pruebas y build mediante GitHub Actions.
- Monitorear drift de datos, calidad, performance del modelo y métricas operativas del servicio.
- Definir y demostrar una política de reentrenamiento y promoción de modelos con criterios cuantitativos y rollback.

4. Caso de uso de Machine Learning

4.1 Variable objetivo

trip_duration_minutes = (tpep_dropoff_datetime - tpep_pickup_datetime) expresado en minutos.

4.2 Variables candidatas disponibles en el momento de predicción

- PULocationID y DOLocationID (origen y destino).
- Hora, día de semana, fin de semana, mes y otras variables derivadas de tpep_pickup_datetime.
- passenger_count, cuando esté disponible y pase reglas de calidad.
- RatecodeID, sujeto a revisión del momento exacto en que se conoce operativamente.
- Características históricas agregadas por ruta/franja horaria, calculadas únicamente con información anterior al viaje.

4.3 Política contra data leakage

No se usarán como predictores variables que materialmente se conocen después de finalizado el viaje, aunque estén presentes en el dataset. Esto incluye tpep_dropoff_datetime, fare_amount, total_amount, tip_amount, payment_type, tolls_amount y cargos calculados al finalizar. trip_distance se tratará con cautela: para un ETA en el instante de inicio, la distancia medida por taxímetro todavía no existe, por lo que no se utilizará en el MVP salvo que se reinterprete el caso de uso.

> **Decisión de diseño:** Esta restricción hace que el proyecto sea más defendible a nivel de Maestría: el modelo se evalúa bajo el mismo conjunto de información que tendría disponible en producción.

5. Arquitectura MLOps propuesta

La solución será modular y separará datos, entrenamiento, serving y observabilidad. Cada etapa deberá poder ejecutarse desde línea de comandos y ser invocable por automatizaciones, evitando que los notebooks sean el único mecanismo de ejecución.

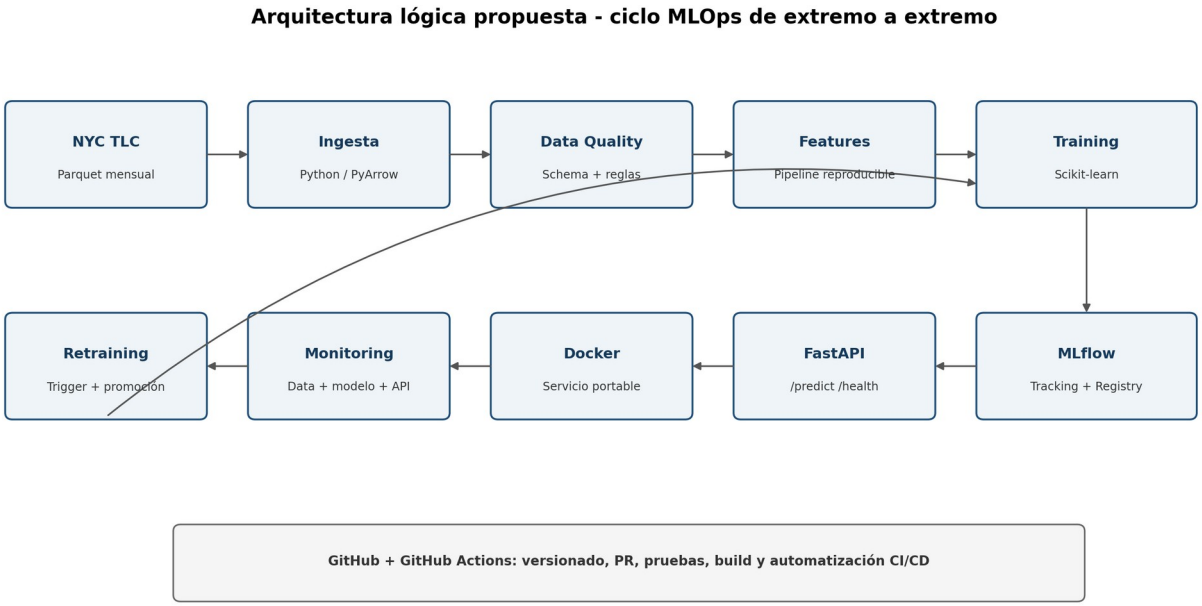


Figura 1. Flujo lógico propuesto del proyecto MLOps.

Capa	Responsabilidad
Fuente	NYC TLC Yellow Taxi Parquet mensual.
Data layer	raw -> validated -> processed/features. Los datos grandes no se versionan directamente en Git.
ML layer	pipeline de entrenamiento, evaluación temporal, MLflow Tracking y Model Registry.
Serving layer	FastAPI carga una versión aprobada del modelo y expone predicción y health checks.
Platform layer	Docker/Docker Compose estandarizan ambientes y servicios.
Automation layer	GitHub Actions ejecuta pruebas, lint, smoke tests y build de imagen.
Observability layer	Evidently para data/model drift; métricas del API para latencia, errores y disponibilidad.
Retraining layer	workflow que incorpora lotes nuevos, reentrena y promueve solo si se cumplen gates.

6. Fases técnicas del ciclo de vida

6.1 Ingesta

Un comando como python -m src.ingestion.download --year 2026 --month 01 debe construir la URL oficial, descargar el Parquet, validar que el archivo exista, calcular hash, registrar tamaño/fecha/origen y almacenarlo en data/raw/2026/01. La tarea debe ser idempotente: repetirla no debe generar inconsistencias.

6.2 Validación y Data Quality Gates

Antes de transformar, el lote pasa por reglas de esquema y negocio. El pipeline termina con estado FAILED cuando una regla crítica se incumple. Las reglas y sus resultados deben quedar persistidos como artefactos para auditoría.

6.3 Preparación y features

Se limpian duraciones imposibles, categorías inválidas y registros fuera de reglas definidas. Se construyen features temporales y de ruta mediante transformadores reproducibles. Todo preprocesamiento requerido en producción debe viajar con el modelo o pipeline serializado.

6.4 Entrenamiento y evaluación

Se utiliza una división temporal, no un split aleatorio puro. Se compara un baseline simple contra modelos de mayor capacidad. Se registra semilla, versión de datos, parámetros, código/commit y métricas.

6.5 Model Registry

El mejor candidato no pasa automáticamente a producción. Se registra en MLflow y se compara contra el champion vigente. La promoción debe ser una acción trazable y reversible.

6.6 API de inferencia

FastAPI expondrá POST /predict, GET /health y GET /model-info. Pydantic validará tipos y rangos. La respuesta debe incluir predicción, unidad, model_version y timestamp de inferencia.

6.7 Docker y ejecución reproducible

El servicio de inferencia y, si se desea, MLflow se ejecutarán con Docker Compose. La imagen debe construirse desde un Dockerfile pequeño, con dependencias bloqueadas y usuario no root cuando sea viable.

6.8 Monitoreo

Se observarán tres planos: datos de entrada, desempeño del modelo cuando aparezca el ground truth y salud del servicio. Los reportes deben persistirse por ventana temporal.

6.9 Retraining

La llegada de un nuevo lote mensual puede activar evaluación de drift. El retraining se ejecuta por criterio, no por simple sustitución del modelo. El nuevo modelo debe superar los gates de calidad y promoción.

7. Data Quality Gates mínimos

Gate	Regla propuesta	Severidad
Esquema	Columnas críticas presentes y tipos compatibles.	Crítico
Volumen	Número de filas > mínimo histórico definido.	Crítico
Fechas	pickup < dropoff; mes del registro consistente con el lote.	Crítico
Duración	0 < duración <= umbral razonable documentado.	Crítico
Duplicados	Tasa de duplicados <= umbral establecido.	Mayor
Nulos	Nulos en PULocationID/DOLocationID por debajo de tolerancia.	Mayor
Categorías	Tasa de valores desconocidos en variables categóricas <= límite.	Mayor
Rangos	passenger_count y códigos dentro de conjuntos/rangos permitidos.	Mayor
Drift de esquema	Nuevas/eliminadas columnas reportadas y clasificadas como compatibles/incompatibles.	Crítico
Trazabilidad	Hash, URL, año/mes y timestamp de ingesta disponibles.	Crítico

8. Protocolo de modelado y métricas

8.1 Modelos

- Baseline: mediana global o mediana por ruta/franja horaria.
- Modelo 1: regresión basada en árboles disponible en scikit-learn (por ejemplo HistGradientBoostingRegressor).
- Modelo 2: RandomForestRegressor o GradientBoostingRegressor como candidato alternativo.
- Opcional: incorporar XGBoost/LightGBM solo si el equipo desea ampliar el stack y puede justificar el costo operativo.

8.2 Métricas

Métrica	Uso
MAE	Métrica primaria; interpretable en minutos.
RMSE	Penaliza con mayor fuerza errores grandes.
MedAE	Robusta ante valores extremos.
P90 error absoluto	Visibiliza experiencia en la cola de errores.
Latencia p95	Métrica operativa de serving, no de exactitud.

9. Monitoreo y política de retraining

Plano	Indicadores
Calidad de entrada	Nulos, rangos, categorías nuevas, volumen, schema drift.
Data drift	Distribuciones de hora, zonas, pasajeros y features derivadas respecto al baseline.
Performance	MAE/RMSE por mes, zona, franja horaria y rutas frecuentes cuando ya exista target.
Servicio	latencia p50/p95, tasa 4xx/5xx, throughput, health check y disponibilidad.
Modelo	versión activa, fecha de entrenamiento, dataset de entrenamiento y edad del modelo.

9.1 Trigger de retraining propuesto

El retraining podrá activarse si ocurre al menos una condición relevante y existe suficiente volumen de datos nuevo. Para evitar reentrenamientos innecesarios, se recomienda exigir persistencia o severidad del cambio.

- Performance: MAE mensual > 115% del MAE de referencia durante dos ventanas consecutivas.
- Drift: PSI > 0.20 (o criterio equivalente documentado) en un conjunto relevante de features críticas.
- Data quality: aparición sostenida de categorías/esquemas nuevos que requieran adaptación del pipeline.
- Cadencia de seguridad: evaluación trimestral del modelo aunque no se dispare un trigger automático.

9.2 Gate de promoción

Un modelo reentrenado será challenger. Solo se promueve si: (a) pasa calidad y pruebas; (b) no presenta regresiones graves por segmentos; (c) mejora el MAE del champion por al menos 2% o

cumple un criterio de no inferioridad previamente definido; y (d) la API supera smoke tests con la nueva versión. Si no cumple, el champion permanece activo.

10. Stack tecnológico

Tecnología	Rol
Python	Lenguaje principal de ingesta, transformación, modelado y automatización.
Pandas / PyArrow	Lectura y procesamiento de Parquet; PyArrow reduce fricción con el formato fuente.
scikit-learn	Pipelines, transformadores, modelos y métricas.
MLflow	Tracking de experimentos, artefactos y Model Registry.
FastAPI + Pydantic	Serving REST y validación de contratos.
Docker / Docker Compose	Ambientes reproducibles y orquestación local de servicios.
Git + GitHub	Versionado de código, issues, pull requests y colaboración.
GitHub Actions	CI/CD: pruebas, lint, build y automatizaciones.
Evidently	Reportes de data drift y model monitoring.
Pytest	Pruebas unitarias, integración y API.

Stack intencionalmente acotado: No conviene introducir Kubernetes, Airflow, Kafka, Terraform y una nube completa solo para “hacerlo más MLOps”. En un proyecto de curso es preferible demostrar bien lineage, automatización, observabilidad, gates y retraining con un stack defendible y ejecutable.

11. Estructura propuesta del repositorio GitHub

```
nyc-yellow-taxi-mlops/
├── .github/workflows/
│   ├── ci.yml
│   └── retraining.yml
├── configs/
│   ├── data.yaml
│   ├── model.yaml
│   └── monitoring.yaml
├── data/
│   ├── raw/.gitkeep
│   ├── validated/.gitkeep
│   └── processed/.gitkeep
├── notebooks/          # solo exploración; no lógica productiva
├── src/
│   ├── ingestion/
│   ├── validation/
│   ├── features/
```

```
│   ├── training/
│   ├── evaluation/
│   ├── monitoring/
│   ├── retraining/
│   └── api/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── api/
├── scripts/
├── reports/
├── Dockerfile
├── docker-compose.yml
├── pyproject.toml
├── Makefile
├── .gitignore
└── README.md
```

Regla de repositorio: archivos Parquet, modelos binarios, mlruns locales, secretos y artefactos de gran tamaño no deben versionarse directamente en Git. El repositorio debe contener código, configuración, pruebas, documentación y punteros/metadatos reproducibles.

12. Flujo CI/CD propuesto

Evento	Pipeline
Pull Request	Lint/formato -> unit tests -> data-pipeline smoke test con muestra -> API tests -> Docker build.
Merge a main	Repite gates; construye imagen versionada por commit/tag; opcionalmente publica en GitHub Container Registry.
Nuevo lote mensual	Ingesta -> DQ gates -> monitoring report -> evaluación de trigger.
Retraining	Train candidates -> MLflow -> compare champion/challenger -> promotion gate -> smoke test -> actualización controlada.
Rollback	La versión previa permanece identificable y desplegable si la nueva falla.

13. Entregables del proyecto

ID	Entregable	Evidencia mínima
D1	Documento de arquitectura y problema	Objetivos, decisiones, diagrama, alcance, riesgos y criterios de éxito.
D2	Repositorio GitHub	Código modular, historial de trabajo, issues/PR y README reproducible.
D3	Pipeline de datos	Ingesta parametrizada, DQ gates,

ID	Entregable	Evidencia mínima
		transformación y evidencia de lineage.
D4	Pipeline de entrenamiento	Baseline + candidatos + validación temporal + MLflow.
D5	API Dockerizada	FastAPI en contenedor con contrato y pruebas.
D6	CI/CD	GitHub Actions ejecutándose con evidencia de runs exitosos y fallidos controlados.
D7	Monitoreo	Reporte de drift/performance y métricas operativas.
D8	Retraining	Demostración de trigger, challenger, gate de promoción y rollback.
D9	Presentación final	Arquitectura, demo, resultados, limitaciones y lecciones MLOps.

14. Criterios de aceptación técnica

- Un integrante distinto al autor original puede clonar el repositorio y reproducir el flujo documentado.
- El pipeline recibe año/mes por configuración o CLI y no depende de rutas hardcodeadas específicas de una máquina.
- Los Data Quality Gates bloquean datos inválidos y producen evidencia del fallo.
- Cada experimento importante en MLflow tiene parámetros, métricas, artefactos y referencia a la versión de datos/código.
- La API valida entradas, retorna versión de modelo y tiene pruebas automatizadas.
- La imagen Docker se construye desde cero mediante CI.
- Existe al menos una simulación de drift y una decisión documentada de retraining/no retraining.
- La promoción de un nuevo modelo está separada del acto de entrenarlo.
- Existe mecanismo documentado de rollback a un modelo anterior.

15. Organización del equipo

Se recomienda que los tres integrantes compartan ownership del repositorio y revisen Pull Requests entre sí. Las responsabilidades pueden rotarse por sprint para evitar silos. Como punto de partida:

Integrante	Foco inicial	Responsabilidades
Bernal Rojas	Data engineering / Data Quality	Ingesta, metadatos, schema validation, tests de datos.

Integrante	Foco inicial	Responsabilidades
Jorge Garnier	Modelado / Experiment tracking	Features, entrenamiento, evaluación, MLflow y model comparison.
Jorge Bermúdez	Serving / Platform / Monitoring	FastAPI, Docker, GitHub Actions, monitoreo y retraining workflow.
Todos	Arquitectura, revisión y documentación	PR review, decisiones de diseño, demo y defensa final.

Importante: La asignación anterior es organizativa, no debe convertir el proyecto en tres piezas desconectadas. Cada integrante debe comprender y poder explicar el ciclo MLOps completo.

16. Plan de trabajo por sprints

Sprint	Foco	Salida
Sprint 1	Arquitectura + EDA técnica	Repo, contratos, descarga reproducible, muestra de datos, ADR inicial.
Sprint 2	Ingesta + calidad	Raw/validated, schema, DQ gates, tests.
Sprint 3	Features + entrenamiento	Baseline, modelos, validación temporal, MLflow.
Sprint 4	Serving + Docker	FastAPI, Pydantic, Docker Compose, tests de API.
Sprint 5	CI/CD + monitoring	GitHub Actions, Evidently, métricas de servicio y reportes.
Sprint 6	Retraining + demo	Trigger, challenger/champion, promoción/rollback, documentación final.

17. Revisión de la propuesta: qué conservar y qué ajustar

Decisión	Elemento	Recomendación
Conservar	Dataset NYC TLC	Excelente para un flujo batch periódico real, con volumen y evolución de esquema.
Conservar	Python + FastAPI + Docker + GitHub	Stack consistente, portable y suficientemente profesional.
Ajustar	“MLOps” como tecnología	Definir MLOps como enfoque; usar MLflow/GitHub Actions/Evidently como herramientas concretas.

Decisión	Elemento	Recomendación
Ajustar	Objetivo del modelo	Fijar un único MVP: duración del viaje. Evitar múltiples targets que diluyan el proyecto.
Ajustar	Split de datos	Usar tiempo real/cronológico; evitar que futuro y pasado se mezclen en train/test.
Ajustar	Retraining	Definir triggers y gates antes de implementarlo; no reentrenar por calendario sin evaluación.
Ajustar	Monitoreo	Separar data quality, drift, performance y operación del API.
Evitar	Complejidad ornamental	No introducir infraestructura distribuida que no agregue evidencia a los objetivos del curso.

18. Decisiones que conviene aprobar antes de implementar

1. Confirmar que el target definitivo será duración del viaje en minutos.
2. Definir cuántos meses se procesarán en el MVP según tiempo y capacidad de cómputo.
3. Confirmar si MLflow y Evidently son aceptables como herramientas adicionales del curso.
4. Definir si la demo final será completamente local con Docker Compose o si se desplegará en un servicio cloud.
5. Acordar umbrales de Data Quality, drift y promoción del modelo que se usarán en la demostración.
6. Definir un criterio común de “Definition of Done” por Pull Request.

Recomendación para la siguiente versión: Una vez validada esta estructura, la Versión 2 debería convertir cada bloque en especificación ejecutable: comandos CLI, contratos Pydantic, archivos YAML de configuración, reglas DQ exactas, experimentos iniciales, Dockerfile, docker-compose.yml, workflows de GitHub Actions y protocolo de monitoreo/retraining.

Referencias y fuentes base

NYC Taxi & Limousine Commission. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

NYC Taxi & Limousine Commission. Data Dictionary - Yellow Taxi Trip Records (18 March 2025). https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf

MLflow. MLflow Documentation. https://mlflow.org/docs/latest/

FastAPI. FastAPI Documentation. https://fastapi.tiangolo.com/

Docker. Docker Documentation. https://docs.docker.com/

GitHub. GitHub Actions Documentation. https://docs.github.com/actions

Evidently AI. Evidently Documentation. https://docs.evidentlyai.com/

Documento de diseño inicial - sujeto a refinamiento durante la implementación.